Directory structure for examples and deployment guide

Introduction

The Certifier has many samples to show how to use it. There are two kinds of examples: the examples for developing applications (in `$CERTIFIER_ROOT/sample_apps/[simple_app, simple_app_for_sev, …])` and examples for developing VM's (as in `$CERTIFIER_ROOT/vm_model_tools/example/scenario1`). We refer to each these directories in context as the `$ENVIRONMENT_DIRECTORY`.

One source of confusion is that the `$ENVIRONMENT_DIRECTORY` in the samples combines actions by entities acting in four separate roles. One goal of this document is to separate what files and programs must be deployed for each role in a real deployment. For the sample_apps, the files for all the roles share the same directory infrastructure, which can be confusing.

The four roles are:

- *Application developer:* This entity compiles and builds the application and utilities.
- *Security domain administrator*: This entity generates keys (including the all-important policy key), run-time files and certifier policy and builds deployment packages[1] for the *Certifier service provider* and *User in a deployed environment*.
- *Certifier service provider*: This entity runs the certifier service.
- *User in a deployed environment*: This is the entity that runs the deployed application or VM.

One core concept is that in the deployed environment, the `$ENVIRONMENT_DIRECTORY` acts as an application directory (like "/Application Support/App…" on a MAC). For the deployed user, the `$ENVIRONMENT_DIRECTORY` need not be protected; all critical files are signed or protected. Sometimes, for some of the roles, the `$ENVIRONMENT_DIRECTORY` is overloaded to provide additional capability (like building an application).

Applications

In the application examples, the *Application developer* compiles the programs in the `$ENVIRONMENT_DIRECTORY` from source files[2]. The developer includes the policy key

[1] The "deployment package" consists of installation scripts for the *Certifier service provider* and *User in a deployed environment* consisting of the required executables, configuration and settings.

[2] The source files for an application need not be in the `$ENVIRONMENT_DIRECTORY`. In fact, this is not the place they would be for the "Mac-like" configuration and state information which is normally in "/Application Support/App" but the compilations are done here in the sample_apps. Also, while the sample_apps generally compile all the certifier files in the `$ENVIRONMENT_DIRECTORY`, the certifier framework also provides a library which can be linked with the application code. Building the app also requires "include files" from the CF

certificate in the image to bind the policy to the app[3]; this is obtained from the *Security domain administrator*.

The *Security domain administrator* generates key files, certs and policy. These files are stored in `$ENVIRONMENT_DIRECTORY/provisioning` as they are generated[4].

When packaging for distribution to a deployed user or certifier service provider, the *Security domain administrator* copies the files needed by the certifier service into `$ENVIRONMENT_DIRECTORY/service`. The *Security domain administrator* will also copy files needed by the deployed application from the provisioning directory into its application file directory, typically, named something like `$ENVIRONMENT_DIRECTORY/app_data`. These will be included in the deployment files for the *Certifier service provider environment* and *User in a deployed environment* respectively.

Both the *Application developer* and the *Security domain administrator* are provisioned with the full Certifier Framework repository for building applications and the key and policy generation programs.

The files deployed to the *User in a deployed environment* never includes the files in the `$ENVIRONMENT_DIRECTORY/provisioning` directory or the `$ENVIRONMENT_DIRECTORY/service` directory; indeed, these have unprotected versions of the private policy key and distributing them completely violates the security promise. But does include some files copied into the "app_data" subdirectory originally generated in provisioning.

In most of the application examples, there are actually two app directories, one for the app acting as a client for the example secure channel and one acting as the server for the example secure channel. In an real deployment package for a *User in a deployed environment*, there would be an `$ENVIRONMENT_DIRECTORY/app_data` for the app acting as TLS server and a `$ENVIRONMENT_DIRECTORY/app_data` for the app acting as a TLS client. The *Security domain administrator* only packages the application executable (which can be anywhere on the executable path but can reasonably remain in `$ENVIRONMENT_DIRECTORY`) and the *app_data* subdirectory, which has some pre-determined files (like the simulated environment

---

source files including those in $CERTIFIER_ROOT when compiling application. This is an example of "overloading" the purpose of some directories to make demonstrating building "sample apps" more easily. In fact, the "VM" model below follows the MAC like paradigm more closely since neither the source file or the resulting executables are in `$ENVIRONMENT_DIRECTORY` in those samples.

[3] But as noted below, this does not happen in the VM case.

[4] Most of the files in the provisioning directory are not needed in either the *User deployed environment* or *Certifier provider environment* as they are "intermediate work products." The necessary files for these two roles are copied into the directories that are provisioned for each of these roles. Once provisioning is complete (and the necessary files are copied to /service, and /app_data), the files in provisioning directory may not ever be needed again

keys)[5]. During installation and operation, in the style of MAC application configuration, the *app_data* subdirectory will be populated with things like machine certs, for example, the ARK, ASK, and VCERT certs on sev.

The *User in a deployed environment* need not contain any other files from the certifier framework. The deployment package for the *User in a deployed environment* can place the executables anywhere but `$ENVIRONMENT_DIRECTORY` is a convenient place. During certification, `$ENVIRONMENT_DIRECTORY/app_data` will contain the policy-store, which protects application specific information in encrypted form.

As a reminder, as deployed in a *User in a deployed environment*, the `$ENVIRONMENT_DIRECTORY` need not be protected, just accessible. For the app samples, the files for all the roles are kept in `$ENVIRONMENT_DIRECTORY`, which may be confusing.

The files in the `$ENVIRONMENT_DIRECTORY/service` are distributed to the *Certifier service provider*; optionally, the `$ENVIRONMENT_DIRECTORY` may contain *simpleserver*, but it need not, provided *simpleserver* is installed in some binary directory that can be accessed. The *Certifier service provider* should not receive the `$ENVIRONMENT_DIRECTORY/app_data` or the `$ENVIRONMENT_DIRECTORY/provisioning` directories.

Neither the *Certifier service provider environment* nor the *User in a deployed environment* need contain any other files from the certifier framework.

An appendix here contains the names of the files typically deployed in the service and app_data directories as well as standard "default" configuration parameters provided to the application that can be changed as with any gflags based applications although applications can add their own. Two additional appendicies provide the same information for VM the VM based utilities below (cf_utility, keyserver and keyclient), in that case, none of the roles will add additional parameters.

VM's

For VM's, the story is very slightly different; the application files for all the roles are kept in `$ENVIRONMENT_DIRECTORY`. However, the programs that interact with the certifier (*cf_utility, keyserver and keyclient*) are not particular to the policy key. Basically, there is no *Application developer* role on a VM.

The *Security domain administrator* is responsible for ensuring all executables in the VM that can use certifier services are measured and that the policy-key cert is also part of the VM

[5] Neither the certifier service nor the deployed user need the certifier utilities, just the app executable.

measurement. This is often accomplished by putting them in initram. This causes a few minor differences.

1. The executables for the certifier (*cf_utility, keyserver and keyclient*) are certainly not kept in the `$ENVIRONMENT_DIRECTORY` and the *Security domain administrator* must ensure that the resolved executable paths and *policy-cert* files refer to their measured locations[6]. With this proviso, the executables can be anywhere.
2. In the *User in a deployed environment,* the policy-store and the cryptstore (where application keys are stored) are in the `$ENVIRONMENT_DIRECTORY`; the remaining application files are stored in `$ENVIRONMENT_DIRECTORY/cf_data`, this directory will contain copies of the ark, ask and vcert certs, in the sev case, the simulated enclave files, and the endorsement cert (in the tpm case).
3. The `$ENVIRONMENT_DIRECTORY/provisioning` and `$ENVIRONMENT_DIRECTORY/service` directories are exactly as in the application setting and should absolutely not be distributed to the *User in a deployed environment*. For most examples, the certifier service machine is fully trusted and contains the private portion of the policy key[7]. The *User deployed environment* never, never has the private portion of the policy key.
4. As before, the *Security domain administrator,* uses the `$ENVIRONMENT_DIRECTORY/provisioning` directory to store generated keys and construct policy.
5. Aside from ensuring the certifier executables come from a trusted directory that was part of the measurement, there is no restriction on where the executables can be stored.
6. The VM utilities (*cf_utility, keyserver and keyclient*) use `$ENVIRONMENT_DIRECTORY` and `$ENVIRONMENT_DIRECTORY/cf_data` for environment storage (for platform certs, for example). The policy_cert file should always be retrieved from the secure measured location and thus will likely have an absolute path name.
7. It is most convenient to keep the application storage hierarchy (`$ENVIRONMENT_DIRECTORY` and `$ENVIRONMENT_DIRECTORY/cf_data`, organized as structured.
8. The `$ENVIRONMENT_DIRECTORY` can be anywhere and need not be protected. It should not, for example, be in initram, since it will change.

Final Note

A protected VM can also host an application service, which treats the VM itself as a secure platform that provides isolate, seal, unseal and attest services to applicatioons. The application

[6] Again, neither the end user nor the certifier service provider need the certifier utilities. The deployed user just needs *cf_utility, keyserver and keyclient.*

[7] There are some examples in sample_app that show how to deploy a certifier service in a CF Framework enforced enclave where the private key is never *"in the clear."* We're not describing that here.

service can provide application services (see *sample_apps/simple_app_under_app_service*) for individual applications and depends on the OS for isolation among such apps. In this case, there will be (at least) three separate `$ENVIRONMENT_DIRECTORY` locations: one for the VM, one for the application service and one for each protected application.

Example of typical files and parameters in different deployment roles

What files are provisioned for a Certifier provider

There are three important files in `$ENVIRONMENT_DIRECTORY/service`, as deployed:

1. The policy private key file specified by the POLICY_KEY_FILE_NAME parameter in the shell scripts. By default, it is"policy_key_file" but of course, changing the flag allows you to change the name.
2. The policy cert file specified by the POLICY_CERT_FILE_NAME parameter in the shell scripts. It contains the self-signed policy cert. By default, "policy_cert_file."
3. The policy file, specified by the POLICY_FILE_NAME flag. By default ,it is “policy.bin.”

In addition, the certifier service, as provisioned, must have the *simpleserver* program.

The certifier service provider directory for VMs has the same files (some defaults are a little different) for VM certification.

What files are provisioned in `$ENVIRONMENT_DIRECTORY/app_data` for a typical application

The `$ENVIRONMENT_DIRECTORY/app_data`, as deployed, should have:

1. The policy cert file (as above)
2. A file called example_app.measurement which contains the app measurement if you use the simulated_enclave.
3. Two files for the simulated enclave: platform_attest_endorsement.bin, and attest_key_file.bin which contain keys and certificates for the simulated enclave.
4. During installation (depending on the platform), several certificate files like “ark_cert.der”, “ask_cert.der”, “vcek_cert.der” and “endorsement_key.der.”

During operation, the policy-store, cleverly named “store.bin,” by default, will be created in the `$ENVIRONMENT_DIRECTORY/app_data` directory (see the flags below). You must also distribute the app, “simple_app.exe,” for example. This can be put anywhere but it is often kept in the `$ENVIRONMENT_DIRECTORY`.

What are the flags and typical default values for a typical application

The following flags are common for applications:

1. The “print_all: specifies the volume of debug output. It is false by default.
2. The operation flag specifies the operation the app is to perform, it is “” by default. Options are: cold-init, get-certified, run-app-as-client, run-app-as-server
3. The “domain_name” flag specifies the security domain name assigned by the *Security domain administrator*, in ssample apps it is "simple-app-home_domain by default.

4. The policy_host flag specifies the ip address of the certifier service, it is "localhost" by default.
5. The policy_port specifies the policy port for the certifier service, it is 8123 by default.
6. The data_dir flag specifies the app data subdirectory name, it is "./app1_data/" by default.
7. The server_app_host specifies the ip address of the application acting as a TLS server, it is "localhost" by default.
8. The server_app_port specifies the port for the app acting as a TLS server, it is 8124 by default.
9. The policy_cert_file specifies the name of the policy cert file, it is "policy_cert_file.bin" by default and is in `$ENVIRONMENT_DIRECTORY/$data_dir/`.
10. The policy_store_file flag contains the name of the policy file store, it is "store.bin" by default and stored in `$ENVIRONMENT_DIRECTORY/$data_dir/`".
11. The public_key_alg flag specifies the asymmetric encryption algorithm used by the app, it is Enc_method_rsa_2048 by default.
12. The symmetric_key_alg flag specifies the symmetric encryption algorithm used by the app, it is Enc_method_aes_256_cbc_hmac_sha256,
13. The platform_file_name flag is the file name for the platform certificate for the simulated enclave, it is "platform_file.bin".
14. The platform_attest_endorsement (defult:"platform_attest_endorsement.bin"), attest_key_file (defult: "attest_key_file.bin") are similar
15. The measurement_file flag specifies the name of the file that contains the app measurement for the simulated enclave, it is "example_app.measurement" by default.
16. ark_cert_file (defult: "ark_cert.der"), ask_cert_file (default: "ask_cert.der"), and "vcek_cert_file" (default "vcek_cert.der") are the names of the files containing those cert (if applicable).

What files are provisioned in `$ENVIRONMENT_DIRECTORY/cf_data` for a typical VM

Files typically present on the VM data directory (default: `$ENVIRONMENT_DIRECTORY/`**)** "as deployed" are:

1. The policy_cert file (as above) as well as the example_app.measurement file for simulated_enclave, the platform_attest_endorsement.bin, and attest_key_file.bin.
2. During installation, the ark, ask, vcek and endorsement certs will also be installed.
3. During initialization, again as above, the policy_store and cryptstore files will be created in `$ENVIRONMENT_DIRECTORY`**.**

What are the flags and typical default values for the VM utilities cf_utility, keyserver and key_client

Most of the flags for cf_utility.exe are the same as for apps including

1. DOMAIN_NAME (default:"datica")
2. POLICY_CERT_FILE_NAME (default:"policy_cert_file.$DOMAIN_NAME")
3. POLICY_STORE_NAME (default: "policy_store.$DOMAIN_NAME)"
4. SYMMETRIC_ENCRYPTION_ALGORITHM (default: "aes-256-gcm")
5. ASYMMETRIC_ENCRYPTION_ALGORITHM (default: "RSA-4096")
6. CRYPTSTORE_NAME (default:"cryptstore/$DOMAIN_NAME")
7. DATA_DIR (default:"./")

8. POLICY_SERVER_ADDRESS (default:"localhost")
9. POLICY_SERVER_PORT (default:"8123")
10. KEY_SERVER_ADDRESS (default:"localhost")
11. KEY_SERVER_PORT (default:"8120")
12. POLICY_SERVER_ADDRESS (default:"localhost")
13. POLICY_SERVER_PORT (default:"8123")
14. PROGRAM_NAME (default:"datica-program")
15. VM_NAME (default:"datica-sample-vm")

There are a couple of more operations for cf_utility.exe:

16. init_trust (default: "true")
17. reinit_trust (default: "true)

There are some additional flags for keyserver and keyclient)

18. KEY_SERVER_ADDRESS (default: "localhost")
19. KEY_SERVER_PORT (default:"8120")
20. get_item (default: false), purpose: get item from cryptstore
21. put_item (default: false), purpose: put item into cryptstore
22. print_cryptstore (default: false), purpose: print cryptstore
23. import_cryptstore (default: false), purpose: import unencrypted cryptstore
24. export_cryptstore (default: false)), purpose: export cryptstore unencrypted"
25. input_format, input file format, (default: serialized-protobuf)
26. output_format, output file format, (default: serialized-protobuf)
27. entry_tag
28. entry_version (generation of key upcpunted when regenerated)
29. entry_type, "key-message-serialized-protobuf",
30. exportable (default: "exportable");
31. keyname
32. duration, what is the key lifetime in hours (default: 24.0 * 365.0)
33. output_file, "output file name");
34. input_file, "input file name");

Its worth consulting `$CERTIFIER_ROOT/vm_model_tools/examples/scenario1/*.sh`, for more background.